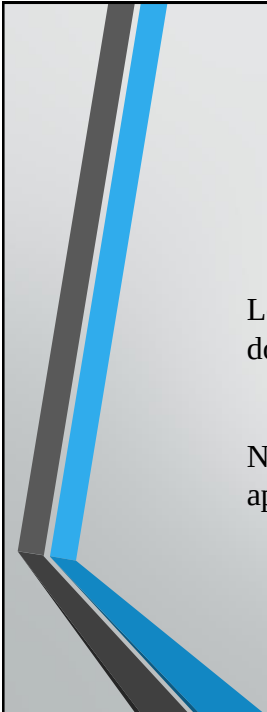




Les Structures Dynamiques Hiérarchiques Les Arbres Binaires



Introduction

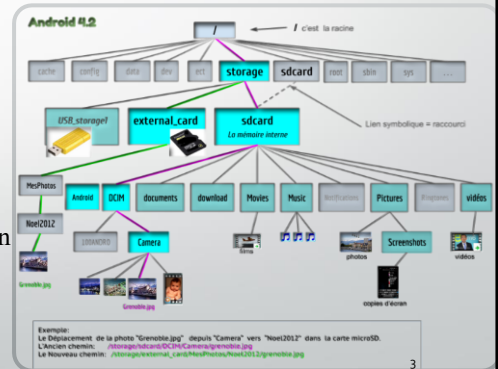
Les structures de données **hiérarchiques** sont très utilisées dans tous les domaines de la vie et particulièrement dans le domaine informatique.

Nous allons nous intéresser dans ce cours en particulier à la structure appelée **Arbre**.

Introduction

Exemples d'arbres :

- Arbre généalogiques des familles
- Sciences Naturelles : arbres des espèces
- Informatique :
 - Arborescence d'une mémoire de stockage
 - Arborescence d'un document (text ou html)
 - Représentation des processus dans la mémoire d'un ordinateur
 - Représentation d'expressions arithmétiques



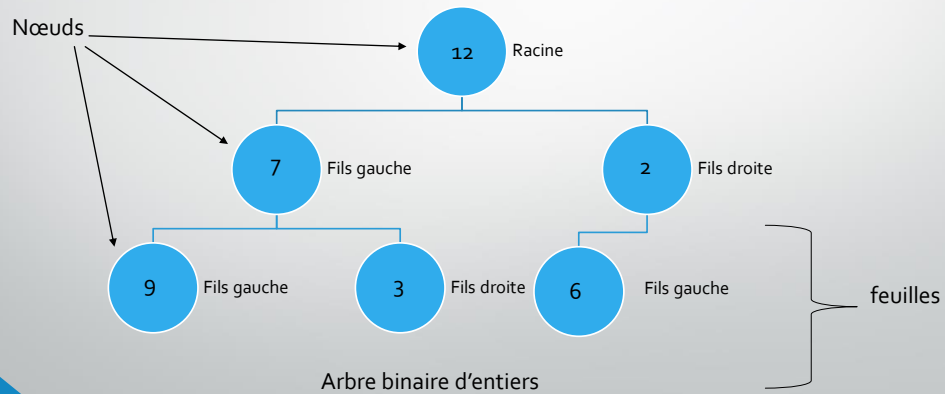
Les Arbres : Définition

- Un arbre est une structure dynamique non linéaire, composé par des éléments reliés entre eux.
- Ces éléments sont des enregistrements appelés aussi **nœud**
- La liaison est assurée à travers les adresses (pointeurs)
- Un arbre est accessible grâce au premier nœud appelé **Racine**.
- Un arbre, s'il n'est pas vide, a une et une seule racine non nulle
- A partir de la racine on peut accéder à n'importe quel autre nœud.
- Chaque nœud a un seul prédécesseur (**père**) et zéro ou plusieurs successeurs (fils)
- Un nœud qui n'a pas de fils est appelé une **feuille**
- La racine est un nœud qui n'a pas de père.

Arbre Binaire

Définition

Un arbre binaire est un arbre où chaque nœud a au maximum deux fils



Implémentation Algorithmique

On suppose qu'on désire implémenter un arbre d'entiers quelconques

Voici une déclaration de types possible :

Type

Nœud = Enregistrement

info : entier

FG : pointeur sur nœud

FD : pointeur sur nœud

Fin nœud

Arbre = pointeur sur Nœud

Var

racine, R, N : Arbre // pointeurs utilisés pour parcourir les nœuds de l'arbre

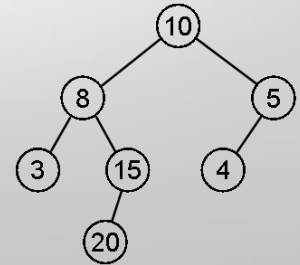
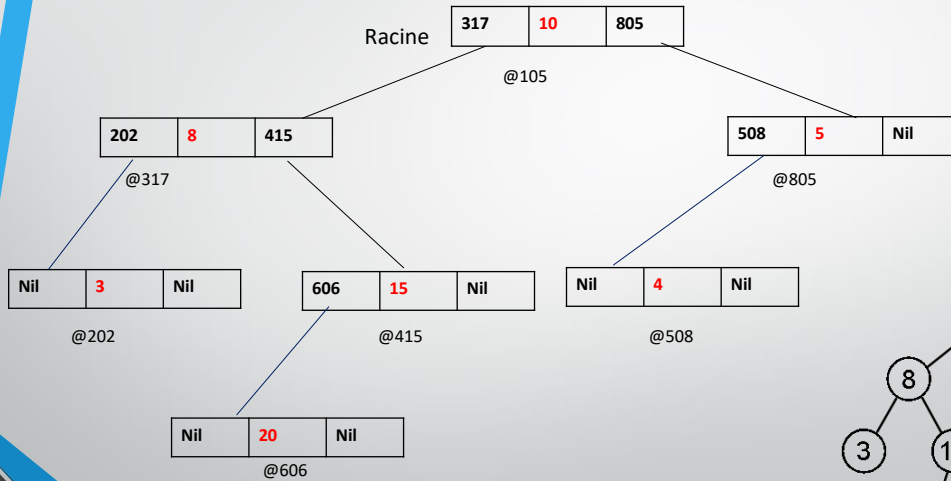
N :

Champ FG	Champ info	Champ FD
----------	------------	----------

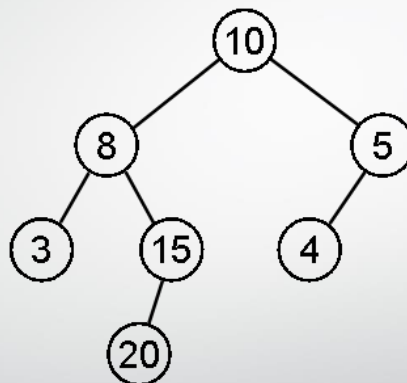
Exemple :

317	10	805
-----	----	-----

Représentation détaillée



Représentation Compacte du même Arbre



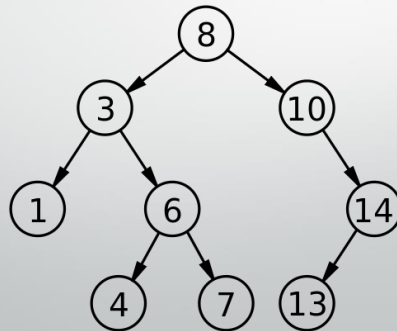
Arbre Binaire de Recherche

Définition

Un arbre binaire de recherche est un arbre binaire tel que pour chaque nœud :

- toutes les valeurs contenues dans le **sous arbre gauche** lui sont **inférieures ou égales**
- Toutes les valeurs contenues dans le **sous arbre de droite** lui sont **supérieures**

Exemple :



Les Algorithmes de parcours

Il existe deux principales manières pour parcourir les nœuds d'un arbre binaire :

- En utilisant un algorithme récursif
- En utilisant un algorithme itératif

Lors du parcours de l'arbre, on peut appliquer un ou plusieurs traitements à un ou plusieurs nœuds. Exemples de traitements :

- Afficher la valeur contenue dans le nœud
- Rechercher une valeur
- Calculer la somme des valeurs contenues dans les nœuds
- Déterminer la taille de l'arbre (nombre des nœuds)

On peut aussi, lors du parcours :

- Supprimer un nœud
- Insérer un nœud

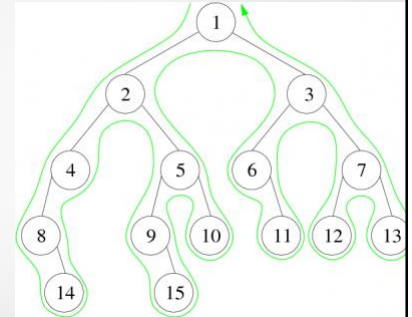
Les Algorithmes de parcours

- Il existe deux façons de parcourir un arbre

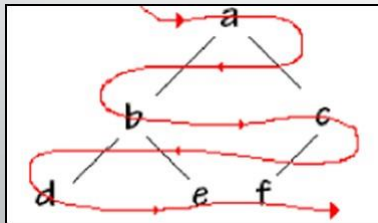
- Parcours en **profondeur**:

- Parcours préfixé
- Parcours infixé
- parcours postfixé

Récurrents



- Parcours en **largeur**



Le parcours en **largeur** est un algorithme itératif.

Algorithmes de parcours Récurrents

Il existe trois principaux algorithmes récurrents de parcours des arbres :

- Algorithme de parcours en **préfixé**

➔ traiter le père puis le sous arbre gauche puis le sous arbre de droite

Père – S.A.G – S.A.D

- Algorithme de parcours en **postfixé**

➔ traiter le sous arbre gauche puis le sous arbre de droite puis le père

S.A.G – S.A.D – Père

- Algorithme de parcours en **Infixé**

➔ traiter le sous arbre gauche puis le père puis le sous arbre de droite

S.A.G – Père – S.A.D

Algorithme de parcours en préfixé avec affichage

Procédure ParcPréfixé (R : Arbre)

Début

Si (R ≠ Nil) alors

 écrire (*R.info)

 ParcPréfixé (*R.FG) // parcourir le S.A.G

 ParcPréfixé (*R.FD) // parcourir le S.A.D

FinSi

Fin

Trace d'exécution du parcours en préfixé Père, S.A.G, S.A.D

1^{er} Appel ParcPréfixé (@105)

① écrire (10)

② ParcPréfixé (@317) → ③ écrire (8)

④ ParcPréfixé (@202) → ⑤ écrire (3) + 2 appels avec des adresses nulles

⑥ ParcPréfixé (@415) → ⑦ écrire (15)

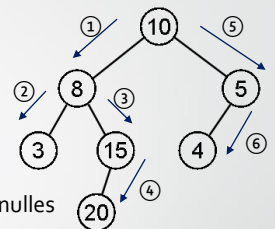
⑧ ParcPréfixé (@606) → ⑨ écrire (20) + 2 appels adresses nulles

⑩ ParcPréfixé (nil) → rien

⑪ ParcPréfixé (@805) → ⑫ écrire (5)

⑬ ParcPréfixé (@508) → ⑭ écrire (4) + 2 appels avec des adresses nulles

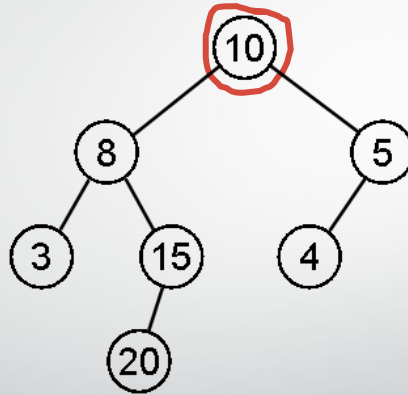
⑮ ParcPréfixé (nil) → rien



Ce qui donne à la fin l'affichage des entiers dans cet ordre : 10 8 3 15 20 5 4

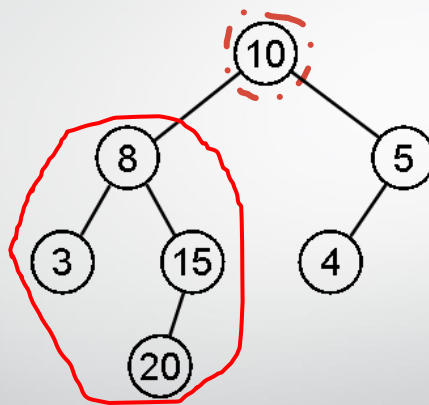
Nœud	adresse
10	105
8	317
5	805
3	202
15	415
4	508
20	606

Trace d'exécution du parcours en préfixé
Racine, S AG, S AD



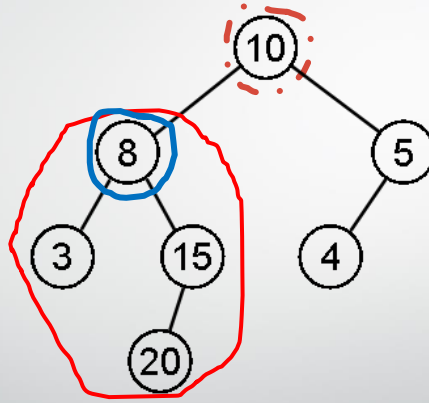
Affichage: 10

Trace d'exécution du parcours en préfixé
Racine, **SAG**, SAD

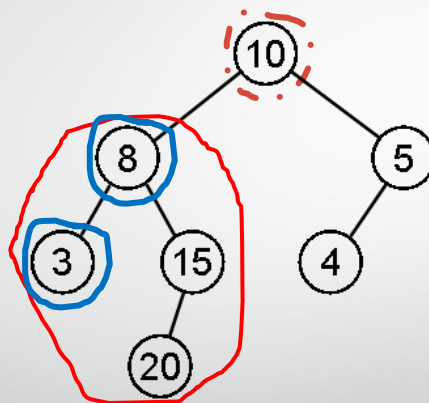


Affichage: 10

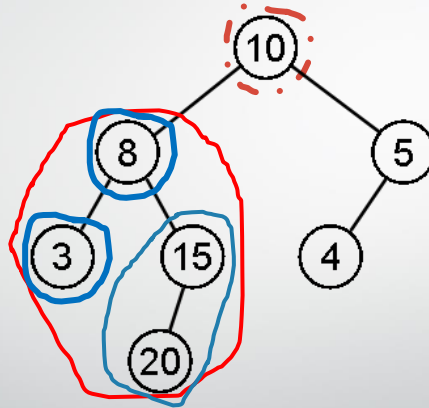
Trace d'exécution du parcours en préfixé
Racine, **SAG**, SAD



Trace d'exécution du parcours en préfixé
Racine, **SAG**, SAD

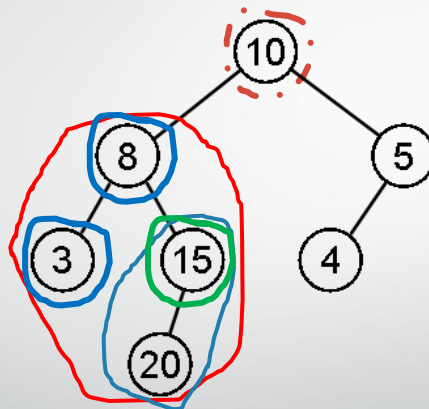


Trace d'exécution du parcours en préfixé
Racine, **SAG**, SAD



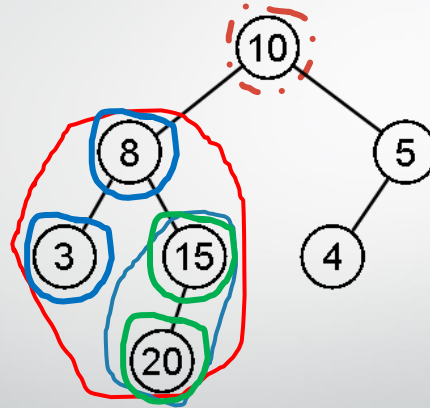
Affichage: 10, 8, 3

Trace d'exécution du parcours en préfixé
Racine, **SAG**, SAD



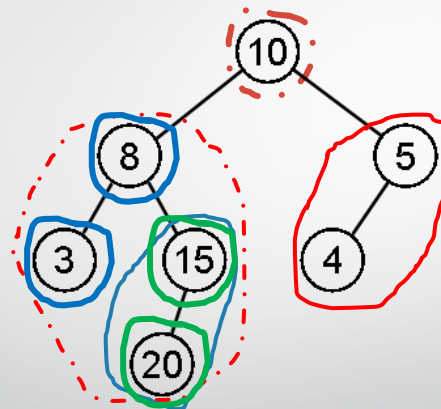
Affichage: 10, 8, 3, 15

Trace d'exécution du parcours en préfixé
Racine, **SAG**, SAD



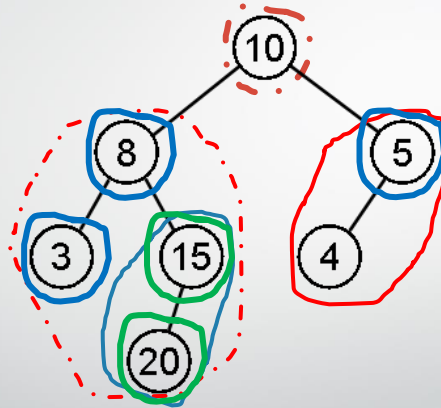
Affichage: 10, 8, 3, 15, 20

Trace d'exécution du parcours en préfixé
Racine, SAG, **SAD**



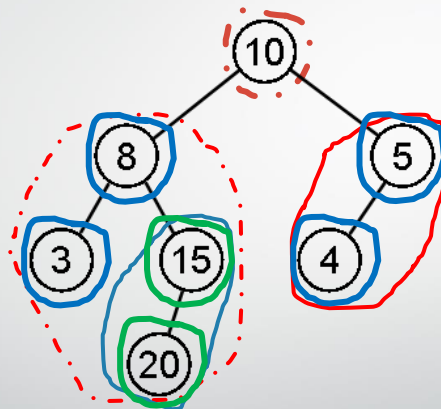
Affichage: 10, 8, 3, 15, 20

Trace d'exécution du parcours en préfixé
Racine, SAG, **SAD**



Affichage: 10, 8, 3, 15, 20, 5

Trace d'exécution du parcours en préfixé
Racine, SAG, **SAD**



Affichage: 10, 8, 3, 15, 20, 5, 4

Algorithme de parcours en Infixé avec affichage

Procédure ParInfixé (R : Arbre)

Début

Si (R ≠ Nil) alors

ParInfixé (*R.FG) // parcourir le S.A.G

écrire (*R.info)

ParInfixé (*R.FD) // parcourir le S.A.D

FinSi

Fin

Trace d'exécution du parcours en Infixé (S.A.G, Père, S.A.D)

↓
ParInfixé (@105)

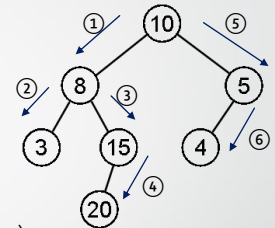
↓
ParInfixé (@317)

→ ParInfixé (@202) → ① écrire (3)
② écrire (8)

ParInfixé(@415) → ParInfixé (@606) → ③ écrire (20)
④ écrire (15)
ParInfixé(nil)

⑤ écrire (10)

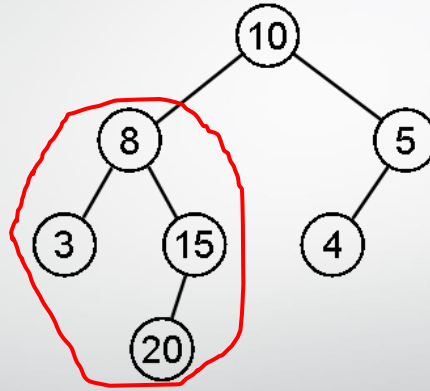
ParInfixé(@805) → ParInfixé (@508) → ⑥ écrire (4)
⑦ écrire (5)
ParInfixé(nil)



Ce qui donne à la fin l'affichage des entiers dans cet ordre : 3 8 20 15 10 4 5

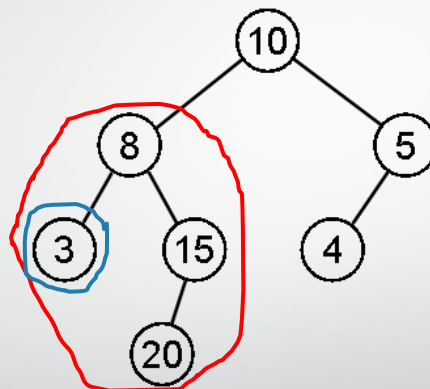
Nœud	adresse
10	105
8	317
5	805
3	202
15	415
4	508
20	606

Trace d'exécution du parcours en infixe
SAG, Racine, SAD



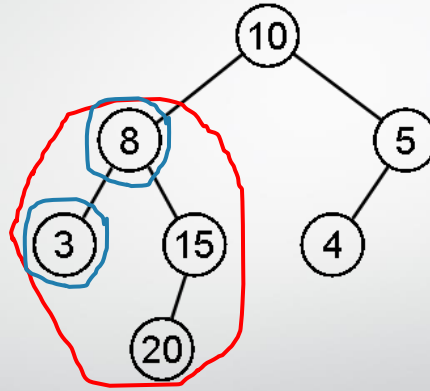
Affichage:

Trace d'exécution du parcours en infixe
SAG, Racine, SAD



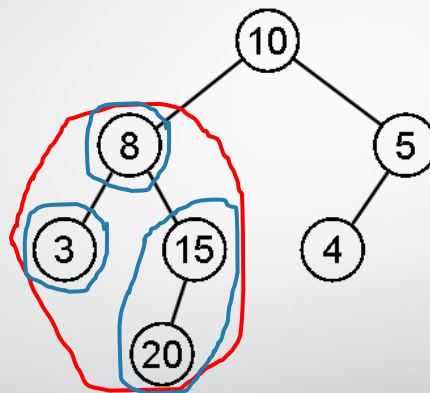
Affichage: 3

Trace d'exécution du parcours en infixe
SAG, Racine, SAD



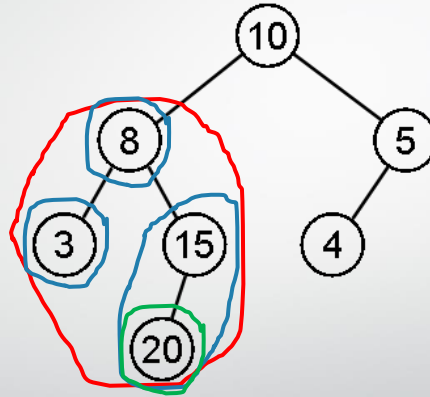
Affichage: 3, 8

Trace d'exécution du parcours en infixe
SAG, Racine, SAD



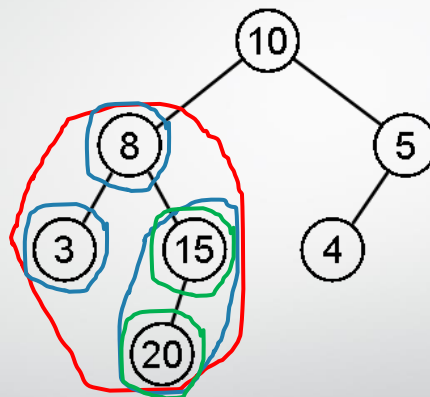
Affichage: 3, 8

Trace d'exécution du parcours en infixe
SAG, Racine, SAD



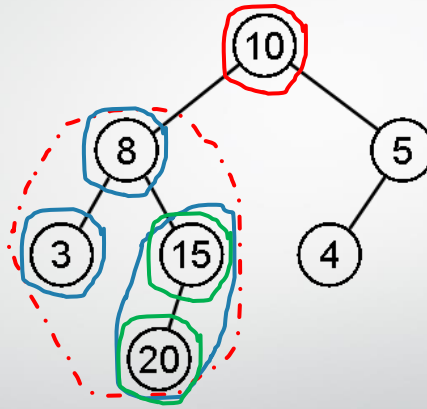
Affichage: 3, 8, 20

Trace d'exécution du parcours en infixe
SAG, Racine, SAD



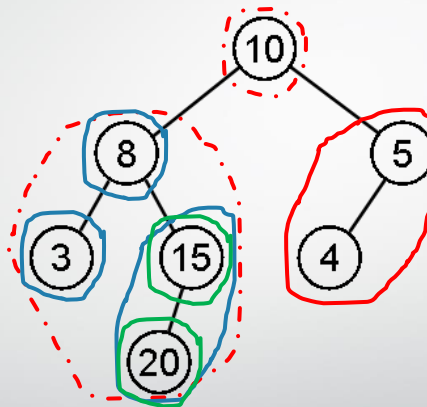
Affichage: 3, 8, 20, 15

Trace d'exécution du parcours en infixe
SAG, **Racine**, SAD



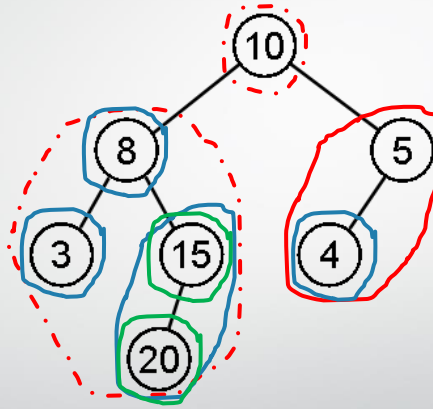
Affichage: 3, 8, 20, 15, 10

Trace d'exécution du parcours en infixe
SAG, Racine, **SAD**



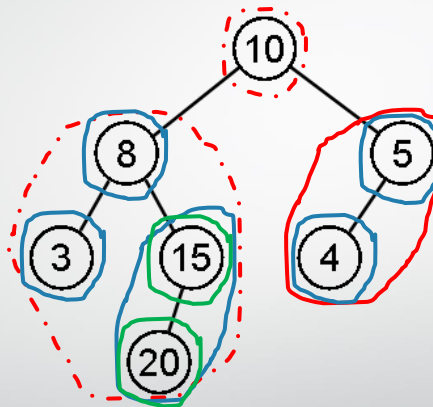
Affichage: 3, 8, 20, 15, 10

Trace d'exécution du parcours en infixe
SAG, Racine, **SAD**



Affichage: 3, 8, 20, 15, 10, 4

Trace d'exécution du parcours en infixe
SAG, Racine, **SAD**



Affichage: 3, 8, 20, 15, 10, 4, 5

Algorithme de parcours en Postfixé avec affichage

Procédure ParcPostfixé (R : Arbre)

Début

Si (R ≠ Nil) alors

ParcPostfixé (*R.FG) // parcourir le S.A.G

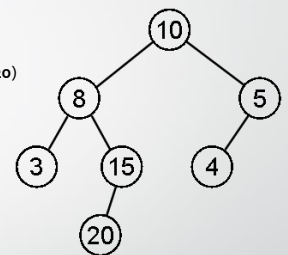
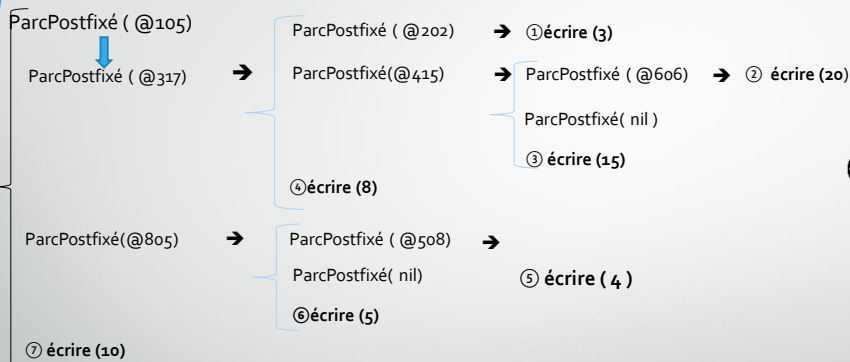
ParcPostfixé (*R.FD) // parcourir le S.A.D

écrire (*R.info)

FinSi

Fin

Trace d'exécution du parcours en Postfixé (S.A.G, S.A.D ,Père)



Nœud	adresse
10	105
8	317
5	805
3	202
15	415
4	508
20	606

Ce qui donne à la fin l'affichage des entiers dans cet ordre : 3 20 15 8 4 5 10

Trace d'exécution du parcours en Infixé dans un arbre binaire de recherche

ParcInfixé(@105)

ParcInfixé (@317)

→ ParcInfixé (@202)

② écrire (3)

ParcInfixé(@415)

→ ① écrire (1)

→ ParcInfixé (@606)

→ ③ écrire (4)

④ écrire (6)

ParcInfixé(719)

→ ⑤ écrire (7)

⑥ écrire (8)

ParcInfixé(@805)

ParcInfixé (nil)

⑦ écrire (10)

ParcInfixé(@508)

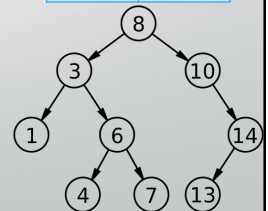
→ ParcInfixé (@984)

→ ⑧ écrire (13)

⑨ écrire (14)

ParcInfixé(nil)

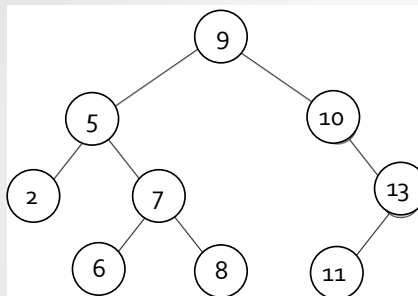
Nœud	adresse
8	105
3	317
10	805
1	202
6	415
14	508
4	606
7	719
13	984



Ce qui donne à la fin l'affichage des entiers dans cet ordre : 1 3 4 6 7 8 10 13 14

Remarque : Le parcours en infixé dans un arbre binaire de **recherche** donne un affichage dans l'ordre **croissant**

Application – Parcours des arbres



Parcours en préordre (ou préfixe) : racine – gauche – droite

9, 5, 2, 7, 6, 8, 10, 13, 11

Parcours terminal (ou postfixe) : gauche – droite – racine

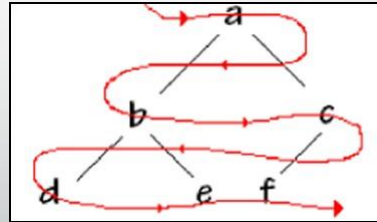
2, 6, 8, 7, 5, 11, 13, 10, 9

Parcours symétrique (ou infixe) : gauche – racine – droite

2, 5, 6, 7, 8, 9, 10, 11, 13

Les Algorithmes de parcours

- Il existe deux façons de parcourir un arbre
 - Parcours en **profondeur**:
 - Parcours préfixé
 - Parcours infixé
 - parcours postfixé
 - **Parcours en largeur**



Parcours en **largeur**

- Un parcours en largeur est un parcours par niveaux
- Tous les nœuds à une profondeur i doivent avoir été visités avant de passer à la profondeur $i + 1$.
- Dans ce parcours on utilise une **file d'attente** qui contient initialement la racine.
- A chaque étape, on élimine l'élément de la file et on ajoute ses successeurs droits et gauches.
- On effectue ce traitement jusqu'à ce que la file soit vide.

Algorithme de Parcours en largeur

Procédure ParcLargeur (racine : Arbre)

Var

temp : Arbre

F : file de pointeur sur Nœud

Debut

Si (racine $\neq$ nil) alors

 Enfiler (F, racine)

 Tant que (NON FileVide (F)) faire

 temp $\leftarrow$ Defiler (F)

 Ecrire (*temp.info)

 Si (*temp.FG $\neq$ nil) alors

 Enfiler (F, *temp.FG)

 Finsi

 Si (*temp.FD $\neq$ Nil) alors

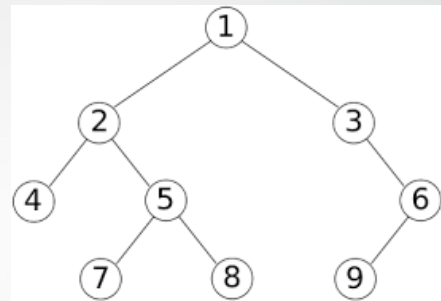
 Enfiler (F, *temp.FD)

 Finsi

 Fin Tantque

FinSi

Fin



F

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

Affichage: 1, 2, 3, 4, 5, 6, 7, 8, 9

43

Suppression d'un nœud d'un arbre binaire

Pour **supprimer** un nœud, on remplace la valeur du nœud par la valeur de l'un de ses fils.

Nous avons trois possibilités. Si le nœud à supprimer:

- A zéro fils: il sera libéré et on lui affecte la valeur Nil
- A un seul fils: on le remplace par son fils non nul
- A deux fils:
 - on fait appel à la fonction TailleArbre(),
 - et on remplace la valeur du nœud par la valeur du fils qui appartient au sous arbre de plus grande taille,
 - puis on fait l'appel récursif de la procédure supprimer dans le sous arbre en question et la valeur à supprimer sera maintenant la valeur contenu dans le fils.

Procédure SuppNoeud(var R : Arbre; x : entier; var Supprim : booléen)

Var

K , Fils: Arbre

Début

Supprim ← faux

Si (R ≠ nil) alors

Si (*R.info = x) alors

 Supprim ← vrai

Si (*R.FG = nil) et (*R.FD = nil) alors /* pas de fils */

 Libérer (R)

 R ← nil

Sinon

Si (*R.FG = nil) alors /* 1 seul fils : fils droit */

 Translation(R, *R.FD)

Sinon

Si (*R.FD = nil) alors /* 1 seul fils : fils gauche */

 Translation(R, *R.FG)

Sinon /* 2 fils */

Si (TailleArbre (*R.FG) ≤ TailleArbre (*R.FD)) alors

 Fils ← *R.FD

Sinon

 Fils ← *R.FG

FinSi

 *R.info ← *Fils.info

 SuppNoeud(Fils, *Fils.info, Supprim)

FinSi

FinSi

FinSi

Sinon /* x non encore trouvé */

 SuppNoeud(*R.FG, x , Supprim)

Si (Non Supprim) alors

 SuppNoeud(*R.FD, x , Supprim)

FinSi

FinSi

Fin

Procédure Translation(var R, Fils :Arbre)

Var

K : ArbreB

Début

K ← R

R ← Fils

 Libérer (K)

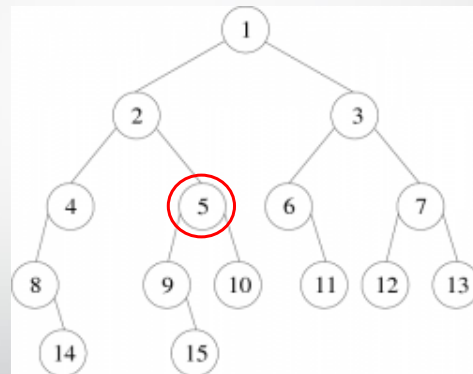
Fin Translation

45

Suppression d'un nœud d'un arbre binaire

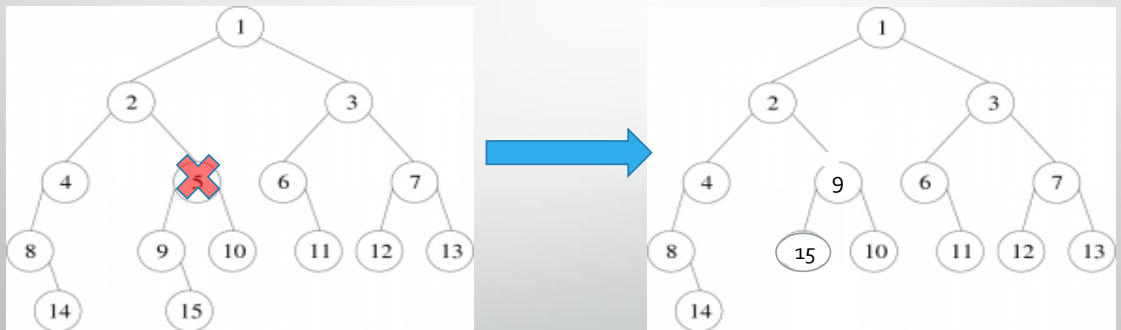
Exemple :

Valeur à supprimer x = 5



Suppression d'un nœud d'un arbre binaire

Exemple :



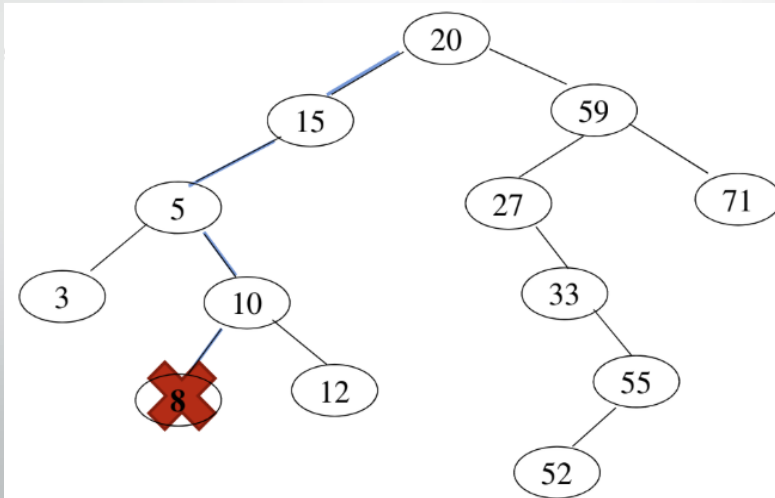
Suppression d'un nœud d'un ABR

Lorsque nous souhaitons supprimer un nœud dans un arbre binaire de recherche nous avons **trois** possibilités. Ce nœud à supprimer:

- A zéro fils
- A un seul fils
- A deux fils

Suppression d'un nœud d'un ABR

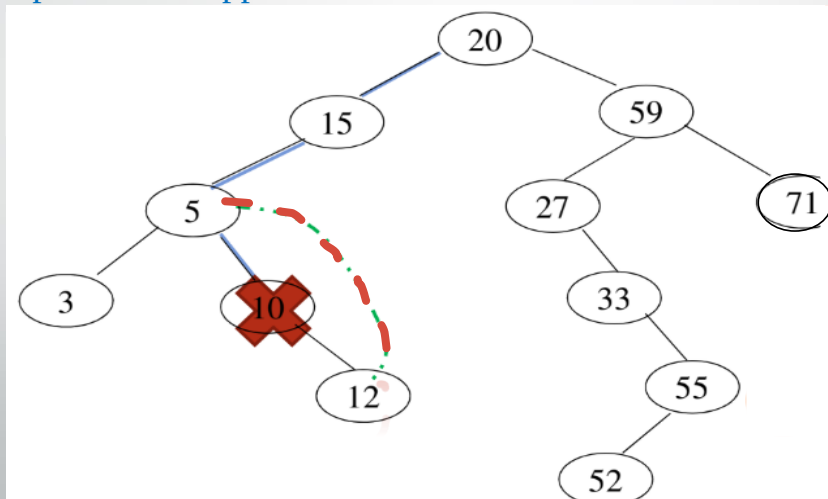
1^{er} cas: Si le nœud à supprimer a zéro fils (c'est une feuille) il suffit de le retirer de l'arbre



49

Suppression d'un nœud d'un ABR

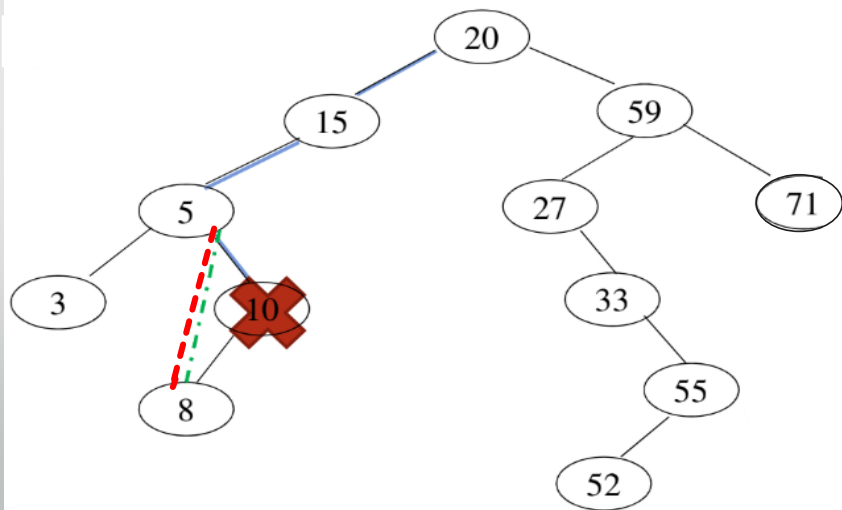
2^{eme} cas: Si le nœud à supprimer a un fils droit, on relie son fils à son père et on supprime le nœud



50

Suppression d'un nœud d'un ABR

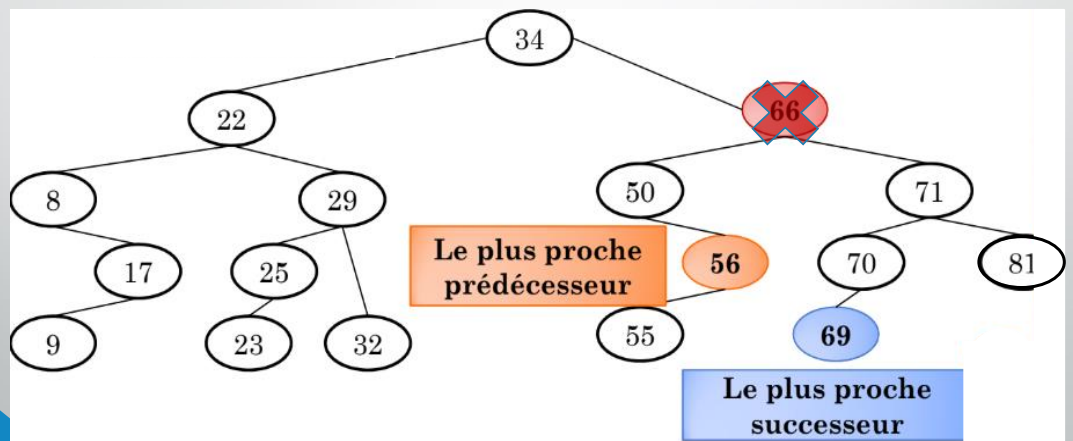
2^{eme} cas: Si le nœud à supprimer a un fils gauche, on relie son fils à son père et on supprime le nœud



51

Suppression d'un nœud d'un ABR

3^{eme} cas: Si le nœud à supprimer a deux fils: on le remplace par son successeur qui est soit le **maximum du SAG** soit le **minimum du SAD**



Procédure SuppNoeudABR(var R : Arbre; x : entier)

Var

Max, Min : entier

Début

Si (R ≠ nil) alors **Si** (*R.info = x) alors **Si** (*R.FG = nil) et (*R.FD = nil) alors

Libérer (R)

R ← nil

Sinon **Si** (*R.FG ≠ nil) alors

Max ← MaxABR(*R.FG)

*R.info ← Max

 SuppNoeudABR(*R.FG, Max) **Sinon**

Min ← MINABR(*R.FD)

*R.info ← Min

 SuppNoeudABR(*R.FD, Min) **FinSi** **FinSi****Sinon** // x non encore trouvé**Si** (*R.info < x) alors

SuppNoeudABR(*R.FD, x)

Sinon

SuppNoeudABR(*R.FG, x)

FinSi**FinSi****FinSi**

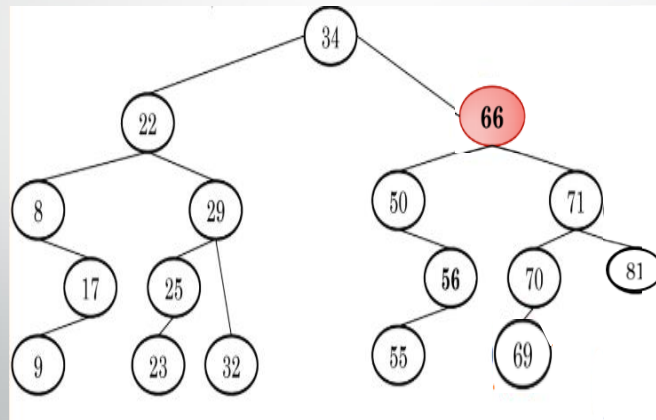
Fin

53

Suppression d'un nœud d'un ABR

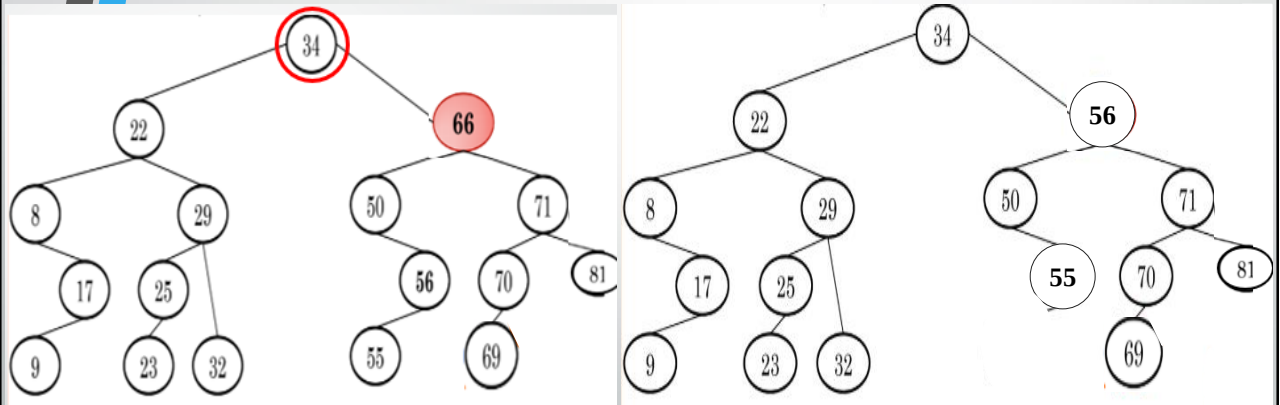
Exemple :

Valeur à supprimer x = 66



Suppression d'un nœud d'un ABR

Exemple :



Insertion / Ajout d'un nœud dans un arbre binaire

L'insertion dans un arbre binaire est délicat si on veut garder un certain équilibre dans l'arbre.

Le principe de l'ajout dans un arbre binaire est le suivant :

- On regarde si le sous arbre gauche est vide, si oui, y insérer le nœud,
- Sinon
- On regarde si le sous arbre droite est vide, si oui y insérer le nœud,
- sinon
- on fait appel à la fonction **TailleArbre** pour les sous-arbres de gauche et de droite
 - on fait l'appel récursif à la procédure d'insertion dans le sous arbre qui contient le moins d'éléments.

Procédure AjoutNoeud_AB (var R : Arbre ; NN : Arbre)

Var TD, TG : entier

Début

Si (R = nil) alorsR $\leftarrow$ NN**Sinon****Si** (*R.FG = nil) alors*R.FG $\leftarrow$ NN**Sinon****Si** (*R.FD = nil) alors*R.FD $\leftarrow$ NN**Sinon**TG $\leftarrow$ TailleArbre(*R.FG)TD $\leftarrow$ TailleArbre(*R.FD)**Si** (TG $\leq$ TD) alors

AjoutNoeud_AB(*R.FG, NN)

Sinon

AjoutNoeud_AB(*R.FD, NN)

FinSi**FinSi****FinSi****FinSi**

Fin

Insertion / Ajout d'un nœud dans un arbre binaire

L'utilisation de la fonction **TailleArbre** pour déterminer de quel côté on va insérer le nœud est un choix qui assure le maintien d'un certain équilibre dans l'arbre.

L'inconvénient de ce choix est qu'il est très gourmand en mémoire surtout lorsque l'arbre est de taille importante, vu les appels récursifs inutilement répétés.

Une autre méthode consiste à utiliser un **algorithme itératif** pour l'ajout de nœud.

```

Procédure AjoutNoeud_It (var R : Arbre ; NN : Arbre)
Var
  node : Arbre , inser : booléen
  F : File de pointeur sur Noeud
Début
  Si (R = Nil) alors
    R ← NN
  sinon
    F ← nil
    inser ← faux
    Enfiler (F, R)
    TantQue (non FileVide (F)) et (inser = faux) Faire
      node ← défiler (F)
      Si (*node.FG = nil ) Alors
        *node.FG ← NN
        inser ← Vrai
      sinon
        si (*node.FD = nil) alors
          *node.FD ← NN
          inser ← vrai
        sinon
          Enfiler (F, *node.FG)
          Enfiler (F, *node.FD)
    FinSi
  Finsi
FinTantQue
FinSi
Fin

```

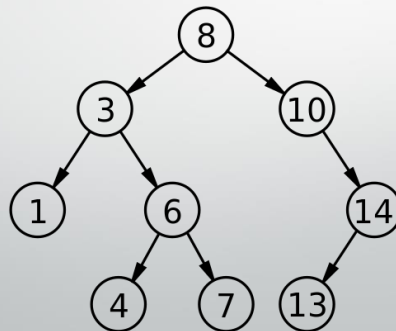
Ajout d'un nœud dans un arbre binaire de recherche

Rappel

Un ABR est un arbre binaire tel que pour chaque nœud :

- Toutes les valeurs contenues dans le **sous arbre gauche** lui sont **inférieures ou égales**
- Toutes les valeurs contenues dans le **sous arbre de droite** lui sont **supérieures**

Exemple :



Procédure AjoutNoeudABR(var R : Arbre; NN : Arbre)

```

Début
  Si (R = nil) alors
    R ← NN
  Sinon
    Si (*NN.info ≤ *R.info) alors // l'ajout sera dans le SAG
      Si (*R.FG = nil) alors
        *R.FG ← NN
      Sinon
        AjoutNoeudABR(*R.FG, NN)
      FinSi
    Sinon // l'ajout sera dans le SAD
      Si (*R.FD = nil) alors
        *R.FD ← NN
      Sinon
        AjoutNoeudABR(*R.FD, NN)
      FinSi
    finSi
  FinSi
Fin

```

61

Ajout d'un nœud dans un arbre binaire de recherche

- Comment créer un ABR à partir d'une suite d'entiers distincts donnés dans un ordre? (par exemple : 15, 23, 8, 12, 1, 14, 5, 20, 18, 3, 6)
- Quelle sera la racine de ABR?
=> c'est le premier entier donné qui prend le rôle de la racine.
- Par la suite, pour chaque nouveau nœud à ajouter on compare sa valeur à **la racine** de l'ARB.
Si val(nouveau nœud) ≤ val(racine) alors on ajoute à gauche
Sinon on ajoute à droite.

Ajout d'un nœud dans un arbre binaire de recherche

- Donner l'ABR obtenu en insérant successivement les entiers suivants dans cet ordre : 15, 23, 8, 12, 1, 14, 5, 20, 18, 3, 6

